

BANCO PLAZA



Tú cuentas

Referencia de Uso: API Banco Plaza

Comprobación de Pagos P2P/P2C

Enero 2026

Versión 4.0.0

Tabla de Contenido

Tabla de Contenido	2
Datos de Dominio	3
Procedimiento para la autenticación de APIs con Oauth 2.0	3
Resumen de Operaciones	9
Pagos P2P/P2C	9
Recurso: /pagos/p2p/v1/v1/pagos/p2p/{id}	10
Argumentos de Entrada	10
Descripción	13
Estructura de Salida	14
Códigos de Respuesta	16

Datos de Dominio

Todos los recursos descritos en el presente documento son relativos al siguiente dominio:

URL para consumo API

Ambiente	Protocolo	Dominio	Puerto
Prueba	HTTPS	openapiqa.bancoplaza.com	N/A
Productivo	HTTPS	openapi.bancoplaza.com	N/A

URL para generación de token de autenticación API

Ambiente	Protocolo	Dominio	Puerto
Prueba	HTTPS	portalapiqa.bancoplaza.com/oauth2/token	N/A
Productivo	HTTPS	portalapi.bancoplaza.com/oauth2/token	N/A

Procedimiento para la autenticación de APIs con OAuth 2.0

OAuth 2.0 es un marco de autorización (Authorization Framework) basado en principios arquitectónicos y estándares de seguridad globales. Su objetivo principal es delegar el acceso a recursos protegidos sin la necesidad de compartir las credenciales directas del usuario. En el ecosistema de WSO2, el endpoint `/oauth2/token` actúa como la puerta central para la gestión de acceso. Su función es realizar el intercambio de credenciales de autorización (como códigos de autorización o credenciales de aplicación) por un Access Token válido.

Requisitos Obligatorios de la Petición

Para garantizar la integridad y seguridad del proceso, todas las solicitudes al endpoint `/oauth2/token` deben contemplar los valores del `client_id` y `client_secret` emitidos por el WSO2 APIM

Definición: CLIENT_ID

Es el identificador único que permite al servidor reconocer qué aplicación específica está solicitando el acceso.

Ejemplo: `client_id`

`VgV9jtz6ozPr_mqwIckJFR5C1oa`

Definición: CLIENT_SECRET

Es la clave secreta o contraseña de la aplicación que verifica su identidad y asegura que solo entidades autorizadas puedan interactuar con la API.

Ejemplo: `client_secret`

`FJ2IciJ3xXW7LhFa2f7oZ_HKNZsa`

Definición: URL TOKEN

Es el recurso de red (punto de acceso) dedicado exclusivamente a la emisión de tokens de acceso. En el modelo de OAuth 2.0, este endpoint actúa como el servidor de autorización.

Características Principales:

- **Método Obligatorio:** Es estrictamente de tipo POST. Esto es así porque el cuerpo de la petición contiene credenciales sensibles y porque la operación no es "idempotente" (estás creando una sesión o recurso nuevo).
- **Seguridad (SSL/TLS):** Al manejar credenciales, este endpoint solo debe estar disponible a través de HTTPS.
- **Procesamiento:** Cuando la URL recibe la petición, el Key Manager de WSO2 realiza tres pasos internos:
 - Valida la autenticidad del `client_id` y `client_secret`.
 - Verifica que la aplicación esté activa.
 - Genera un token firmado con un tiempo de expiración definido.

Procedimiento para la solicitud de token

Antes de consumir el endpoint, necesitas las credenciales emitidas por Banco Plaza.

- `client_id`: Tu identificador único
- `client_secret`: Tu contraseña de aplicación.

La solicitud se envía mediante un método POST a la URL del consumo del token. Este paso es el intercambio formal de identidad por acceso a continuación se describen la información de acuerdo con el ambiente.

URL de consumo del token

Ambiente	Protocolo	Dominio	Puerto
Prueba	HTTPS	portalapiqa.bancoplaza.com/oauth2/token	N/A
Productivo	HTTPS	portalapi.bancoplaza.com/oauth2/token	N/A

Para realizar el proceso de consumo de las APIs se requiere la emisión del token, para ello se ha incorporado un script basado en el concepto de encadenamiento de peticiones, su funcionalidad es obtener un token y posteriormente enviarlo como parámetro al header: Authorization.

A continuación, se mencionan los parámetros a considerar remplazar su valor para consumo de las APIS en un código de ejemplo en Javascript y la explicación de este

Los valores que se deben de cambiar son:

tokenUrl = Url del token

Ejemplo: <https://portalapiqa.bancoplaza.com/oauth2/token>

clientId = valor del client_id

Ejemplo: N6nAqhrTrPd7rwjAazbeZ4EWpWMa

clientSecret = valor del client_secret

Ejemplo: rtUiIwvA4SWSMH8anD2eUdREQ6Aa

```
Pre-request 1 // 1. Definir los datos para la solicitud del token
2 const tokenUrl = 'https://portalapiqa.bancoplaza.com/oauth2/token';
Post-response 3 const clientId = 'N6nAqhrTrPd7rwjAazbeZ4EWpWMa'; //boradado qa
4 const clientSecret = 'rtUiIwvA4SWSMH8anD2eUdREQ6Aa'; //boradado qa
```

Nota: cada uno de los recursos se deberá hacer el remplazo de la información que anteriormente se menciona

La función principal del script es mostrar el proceso a seguir para automatizar la obtención de un token de acceso (OAuth 2.0) antes de ejecutar la petición principal de la colección.

A continuación, se muestra el desglose paso a paso:

1. Configuración de Credenciales

Se definen las variables necesarias para identificarse ante el servidor de Banco Plaza:

- **URL del Token:** La dirección del servidor de autorización (tokenUrl).

```
Pre-request 1 // 1. Definir los datos para la solicitud del token
2 const tokenUrl = 'https://portalapiqa.bancoplaza.com/oauth2/token';
```

- **Credenciales:** Define el clientId y el clientSecret.

```
Post-response 3 const clientId = 'N6nAqhrTrPd7rwjAazbeZ4EWpWMA'; //boradado qa
4 const clientSecret = 'rtUIiwvA4SWSMH8anD2eUdREQ6Aa'; //boradado qa
```

- **Codificación:** Convierte estas credenciales a un formato **Base64** usando la función btoa(). Esto es un requisito estándar para el encabezado de "Authorization: Basic".

```
7 const base64Auth = btoa(clientId + ":" + clientSecret);
```

2. Definición del Objeto de Petición (requestOptions)

Se construye la estructura de la solicitud HTTP que se enviará:

- **Método:** POST.
- **Headers:** Configura el tipo de contenido como application/x-www-form-urlencoded e incluye el token de autorización básica que se generó en el paso anterior.
- **Cuerpo (Body):** Define que el flujo de autenticación es de tipo client_credentials, que es el usado para comunicación de servidor a servidor (sin intervención de un usuario final).

```
9 const requestOptions = {
10   url: tokenUrl,
11   method: 'POST',
12   header: {
13     'Authorization': 'Basic ' + base64Auth,
14     'Content-Type': 'application/x-www-form-urlencoded'
15   },
16   body: {
17     mode: 'urlencoded',
18     urlencoded: [
19       { key: 'grant_type', value: 'client_credentials' }
20     ]
21   }
22 };
23
```

3. Ejecución de la Petición (pm.sendRequest)

El script envía la solicitud de forma asíncrona:

- Manejo de Errores: Si algo sale mal (problemas de red, URL incorrecta), imprime el error en la consola.
- Procesamiento de Respuesta: Si la petición es exitosa, convierte la respuesta del servidor en un objeto JSON para poder leerla.

```

25 pm.sendRequest(requestOptions, (err, response) => {
26   if (err) {
27     console.log("Error obteniendo el token:", err);
28   } else {
29     const jsonResponse = response.json();
30     const accessToken = jsonResponse.access_token;
31
32     // 3. Guardar el token en una variable de colección
33     // "current_token" es el NOMBRE de la etiqueta en Postman
34     pm.collectionVariables.set("current_token", accessToken);
35
36     console.log("Token actualizado correctamente");
37
38     // CORRECCIÓN: Usamos accessToken que es la variable de JS
39     console.log("Token: " + accessToken);
40   }
41 });

```

4. Extracción y Almacenamiento del Token

Esta es la parte más importante para el flujo de trabajo:

- Extracción: Toma el valor del campo access_token que devolvió el banco.
- Persistencia: Guarda ese valor en una variable de colección llamada current_token.
- Log: Imprime el token en la consola para que el desarrollador pueda verificar que se recibió correctamente.

```

34 pm.collectionVariables.set("current_token", accessToken);
35
36 console.log("Token actualizado correctamente");
37
38 // CORRECCIÓN: Usamos accessToken que es la variable de JS
39 console.log("Token: " + accessToken);
40 }
41 });

```

Resumen de Operaciones

Pagos P2P/P2C

Requiere autenticación	Endpoint	Método HTTP	Permisos
Sí	/pagos/p2p/v1/v1/pagos/p2p/{id}	GET	READ

Recurso: /pagos/p2p/v1/v1/pagos/p2p/{id}

Argumentos de Entrada

A continuación, se listan los argumentos de entrada que se utilizarán para definir criterios de búsqueda especializados. Todos los argumentos de entradas deben ser indicados en el QueryString.

	Argumento	Tipo	Descripción
Argumentos Requeridos	id	String	<i>Identificación.</i> Se refiere al documento de identidad del cliente del que se desea consultar la información. Debe incluir el tipo de persona. Por ejemplo, para persona natural el valor sería V13759368. En contraste, para un comercio sería J00378944781. Longitud de doce (12) caracteres.
	canal	String	<i>Canal.</i> Código que busca clasificar el origen de la transacción. Los posibles valores son los siguientes: • "20" para POS • "21" para MERCHANT • "22" para VPOS • "23" para BOTON DE PAGO • "24" para BILLETERA DIGITAL Entre otros
Argumentos Opcionales	acc	Integer	<i>Acción.</i> Se utiliza para indicar la dirección de las transacciones P2P/P2C a consultar. • Cero (0) para transacciones entrantes. • Uno (1) para transacciones salientes. • Mayor que uno (1) para todas las transacciones. En el caso de que el argumento esté ausente, se asume que su valor es igual a cero (0).
	fi	String	<i>Fecha Inicio.</i> Se utiliza para filtrar por todas aquellas transacciones P2P/P2C cuya fecha sea igual o mayor al valor indicado en el argumento. El formato para el argumento es el ISO 8601. Ej.: YYYY-MM-DD.

Si el valor del argumento se encuentra por debajo del límite definido para el servicio P2P/P2C, entonces *fi* toma el valor establecido por el límite.

El valor del límite del servicio es de seis (2) meses, pero puede variar sin previo aviso.

Este argumento se puede usar junto con el argumento *ff* para definir filtros de rangos de fechas.

En caso de que el argumento esté ausente, asume el valor del límite del servicio, salvo que el argumento *ff* también se encuentre ausente, en cuyo caso asumirá el valor de la fecha corriente.

<i>ff</i>	String	<i>Fecha Fin.</i> Se utiliza para filtrar por todas aquellas transacciones P2P/P2C cuya fecha sea igual o menor al valor indicado en el argumento.
-----------	--------	--

El formato para el argumento es el ISO 8601. Ej.: YYYY-MM-DD.

Si el valor del argumento es mayor a la fecha corriente, entonces *ff* tomará el valor de la fecha corriente.

Este argumento se puede usar junto con el argumento *fi* para definir filtros de rangos de fechas.

El en caso de que el argumento esté ausente, asume el valor de la fecha corriente.

<i>tlf</i>	String	<i>Teléfono.</i> Se utiliza para filtrar por el número telefónico dadas las siguientes condiciones:
------------	--------	---

- Si el argumento *acc* es cero (0), entonces retorna todas aquellas transacciones recibidas por el número telefónico *tlf*.
 - Si el argumento *acc* es uno (1), entonces retorna todas aquellas transacciones enviadas al número telefónico *tlf*.
 - Si el argumento *acc* es mayor a uno (1), entonces retorna todas aquellas transacciones recibidas por y enviadas al número telefónico *tlf*.
-

En el caso de que el argumento esté ausente, no se asume ningún valor por omisión, simplemente es ignorado como filtro.

El formato que debe cumplir el parámetro de teléfono es como sigue: [código de país] + [Código Operadora] + [Número Teléfono].

Ejemplo: 00582122578567.

No debe llegar ningún carácter especial, tal como paréntesis, guiones o puntos.

tlfa	String	<i>Teléfono Afiliado.</i> Se refiere al número de teléfono asociado a la afiliación del propietario de las llaves.
------	--------	--

En el caso de que el argumento esté ausente, se asume el valor por omisión, que es el informado al equipo de Banco Plaza al momento de suscribirse al servicio de Open Bank.

El formato que debe cumplir el parámetro de teléfono es como sigue: [código de país] + [Código Operadora] + [Número Teléfono].

Ejemplo: 00582122578567.

No debe llevar ningún carácter especial, tal como paréntesis, guiones o puntos.

horaIni	String	<i>Hora Inicio.</i> Se utiliza para filtrar por todas aquellas transacciones P2P/P2C cuya hora sea igual o mayor al valor indicado en el argumento.
---------	--------	---

El formato para el argumento es tiempo militar HHMMSSMS. Ejemplos: 15350501.

En tiempo militar, las horas se representan de 00 a 23, donde 00:00 (medianoche) es el inicio del día y 23:59 (11:59) es el último minuto del día.

Este argumento se puede usar junto con el argumento “horaFin” para definir filtros de rangos de tiempo.

En caso de que el argumento esté ausente, asume el valor del límite del servicio, salvo que el argumento “horaFin” también se encuentre

		ausente, en cuyo caso asumirá el valor para el día completo.
horaFin	String	Hora Fin. Se utiliza para filtrar por todas aquellas transacciones P2P/P2C cuya hora sea igual o menor al valor indicado en el argumento. El formato para el argumento es tiempo militar HHMMSSMS. Ejemplos: 15350501. En tiempo militar, las horas se representan de 00 a 23, donde 00:00 (medianoche) es el inicio del día y 23:59 (11:59) es el último minuto del día. Este argumento se puede usar junto con el argumento "horalni" para definir filtros de rangos de tiempo. El en caso de que el argumento esté ausente, asume el valor de la hora corriente.
Permisos	READ	
Método HTTP	GET	
HTTP Status Code		200 Accepted 204 No Content 400 Bad Request 401 Unauthorized 500 Internal Server Error

Descripción

Permite realizar consultas sobre el historial de transacciones P2P/P2C del cliente que invoca el recurso, haciendo uso de los distintos argumentos o filtros dispuestos para tal fin. La identificación del cliente se determina a través del client_id y el client_secret.

Algunos ejemplos de uso podrían ser los siguientes:

QueryString	Resultado Esperado
Omitida	Retorna las transacciones recibidas para el día de hoy.
?tlf=[valor]	Retorna las transacciones recibidas para el día de hoy originadas por el número de teléfono especificado en [valor].
?tlf=[valor]&acc=1	Retorna las transacciones enviadas al número de

	teléfono especificado en [valor] el día de hoy.
?fi=[value]&ff=[value]	Retorna las transacciones recibidas en el rango de fecha especificado para fi y ff.
? horaIni=[value]&horaFin=[value]	Retorna las transacciones recibidas en el rango de horas especificado para fi y ff.
? id_Pago=[value]	Retorna las transacciones recibidas cuyo ordenante se ala identificación del id_pago.

Estructura de Salida

Se refiere a los datos que componen la estructura de salida para el recurso. Básicamente, es la respuesta a la petición. Se describe en la siguiente tabla:

Campo	Tipo	Descripción
codigoRespuesta	String	<i>Código de Respuesta.</i> Se refiere a un código de cuatro (4) dígitos que identifica la respuesta generada por el servicio. Este campo será agregado a la cabecera de la respuesta HTTP.
descripcionCliente	String	<i>Descripción para el Cliente.</i> Es la descripción asociada al <i>Código de Respuesta</i> diseñada para ser mostrada o desplegada a la capa front. Este campo será agregado a la cabecera de la respuesta HTTP.
descripcionSistema	String	<i>Descripción del Sistema.</i> Es la descripción asociada al <i>Código de Respuesta</i> diseñada para albergar información técnica sobre la respuesta a la petición. Este campo será agregado a la cabecera de la respuesta HTTP.
fechaHora	Datetime	<i>Fecha y Hora.</i> Es una marca de tiempo o timestamp del momento exacto en que se entrega la respuesta a la petición. Este campo será agregado a la cabecera de la respuesta HTTP.
pagos	List<Pago>	<i>Lista de Pagos.</i> Se refiere a una lista de objetos

		JSON que contiene la información de cada uno de los pagos que cumplieron con el criterio de búsqueda suministrado.
cantidadPagos	Integer	<i>Cantidad de Pagos.</i> Se refiere a la cantidad de objetos Pago dentro de la lista pagos.
Este campo será agregado a la cabecera de la respuesta HTTP.		

Dado que pagos es una lista de objetos Pago, es necesario detallar la estructura de dicho objeto. En la siguiente tabla se listan los atributos que lo componen:

Campo	Tipo	Descripción
accion	String	<i>Acción.</i> Indica si se trata de un pago recibido (R) o enviado (P).
banco	String	<i>Banco.</i> Código de cuatro (4) dígitos que identifica a la entidad bancaria ordenante de la transacción si acción es igual a “R” o a la beneficiaria si acción es igual a “P”.
telefonoCliente	String	<i>Teléfono Cliente.</i> Numero de teléfono del ordenante de la transacción si acción es igual a “R” o del beneficiario si acción es “P”.
telefonoAfiliado	String	<i>Teléfono Afiliado.</i> Número de teléfono del cliente afiliado al servicio de P2P/P2C en Banco Plaza. Es indistinto del valor de acción.
monto	Double	<i>Monto.</i> Se refiere al valor de la transacción P2P expresado en bolívares.
fecha	String	<i>Fecha.</i> Se trata de la fecha de la transacción en el siguiente formato: YYYYMMDD.
hora	String	<i>Hora.</i> Se trata de la hora de la transacción en el siguiente formato: HHMM
referencia	String	<i>Número de referencia.</i> Código generado por el sistema para identificar de forma unívoca el pago dentro del sistema interbancario.
concepto	String	<i>Concepto.</i> Descripción que el cliente le da la transacción.

A continuación, se muestra un ejemplo de una respuesta para este recurso:

Response Header

HTTP/1.1 200 OK

codigoRespuesta: 0000

descripcionCliente: Transaccion Exitosa

descripcionSistema: Transaccion Exitosa

fechaHora: 2019-11-28 09:12:45

Body

```
{
  cantidadPagos: 2
, "pagos": [
  {
    "accion": "R"
  , "banco": "0102"
  , "telefonoCliente": "00584247835644"
  , "telefonoAfiliado": "00584126389938"
  , "monto": 1000.00
  , "fecha": "20191027"
  , "hora": "1635"
  , "referencia": "872675937"
  , "motivo": "Recarga servicio"
  },
  {
    "accion": "R"
  , "banco": "0134"
  , "telefonoCliente": "00584141194765"
  , "telefonoAfiliado": "00584126389938"
  , "monto": 35000.00
  , "fecha": "20191028"
  , "hora": "1746"
  , "referencia": "872677983"
  , "motivo": "Pago comercio"
  }
]
}
```

Códigos de Respuesta

La lista de los posibles Códigos de Respuesta se presenta a continuación:

Código de Respuesta	Descripción Sistema	Tipo de Resultado	HTTP Status Code
---------------------	---------------------	-------------------	------------------

0000	TRANSACCIÓN EXITOSA	Transacción exitosa.	200
0002	PARÁMETRO [parametro] OBLIGATORIO	Error de sistema.	400
E001	CLIENTE NO REGISTRADO	Validación de sistema	500
E002	TELEFONO NO REGISTRADO	Validación de sistema	500
E003	REGISTRO NO EXISTE	Validación de sistema	204
E024	CLIENTE BLOQUEADO	Validación de sistema	500
N002	CEDULA/RIF NO NUMERICA	Validación de sistema	400
N003	TELEFONO NO NUMERICO	Validación de sistema	400
N004	TELEFONO 1 NO NUMERICO	Validación de sistema	400
N005	TELEFONO 2 NO NUMERICO	Validación de sistema	400
V008	TELEFONO NO VALIDO	Validación de sistema	400
0096	ERROR EN SISTEMA	Error de sistema	500
A001	FIRMA DIGITAL INVÁLIDA	Error de sistema	400
A002	FIRMA DIGITAL VENCIDA	Error de sistema	500
A003	RECURSO NO AUTORIZADO	Validación de sistema	401
A004	API-KEY INVÁLIDA O REVOCADA	Validación de sistema	401
0095	HORA INICIO INVALIDA	Error de sistema.	400
0094	HORA FIN INVALIDA	Error de sistema.	400